Napredni algoritmi i strukture podataka

Predavanje 1: Zašto ove dvije ideje zajedno.

dr.sc.tech. Kristijan Poje

Što je algoritam?

- Postupak korak-po-korak koji ulaz pretvara u izlaz
- ...ali to nije dovoljno da bismo ga u ovom kolegiju nazvali "algoritmom"
- Vrijedi tek kada o njemu možete tvrditi dvije stvari:
  1. **Ispravnost** — radi za *svaki* dopušteni ulaz, ne samo za one koje ste isprobali
  2. **Cijena** — što potroši da to postigne, iskazano dovoljno precizno da netko drugi to može provjeriti

Što je struktura podataka?

- Nije "spremnik" niti "način pohrane podataka"
- To je **raspored podataka koji određene zaključke čini jeftinima za izvući**
- I, nužno, druge zaključke čini skupima — nijedna struktura ne daje sve besplatno

Veza: zašto se ovo dvoje uvijek predaje zajedno

- Cijena algoritma nikada nije samo činjenica o kodu — to je činjenica o **rasporedu koji smije pretpostaviti**
- Poznati "$\log_2 n$" binarnog pretraživanja nije domišljat kôd. To je cijena pretpostavke da je polje već sortirano.
- Uklonite sortirani poredak i binarno pretraživanje i dalje *radi*. Samo prestaje biti *ispravno*.
- **Jamstvo živi u strukturi. Algoritam ga samo unovčava.**

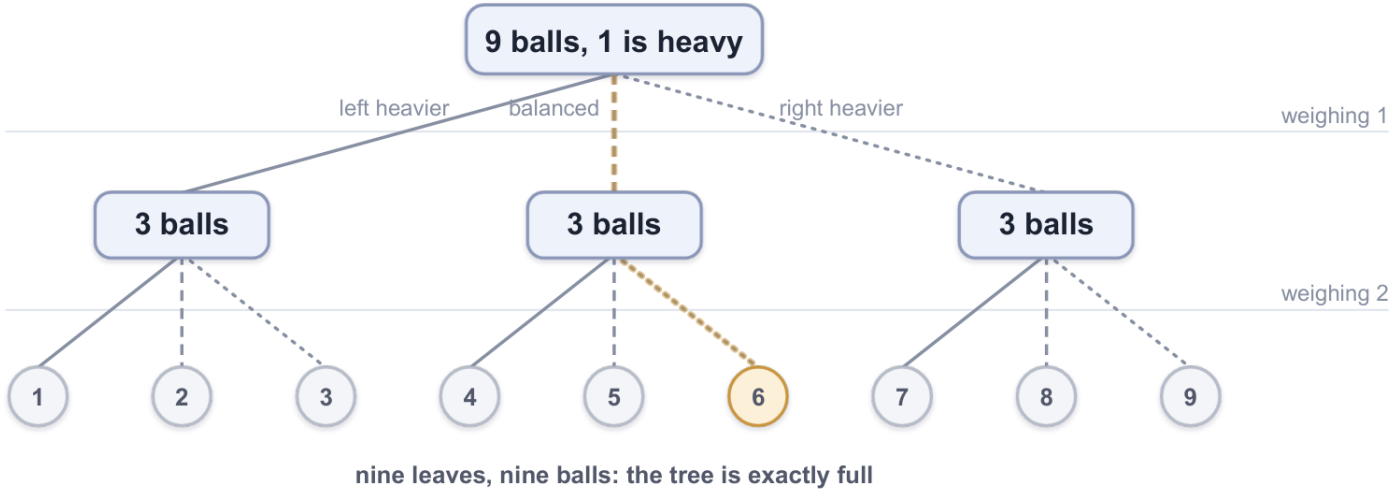
Zašto ovo učimo / plan za danas

- Danas = Poglavlje 1: rječnik koji će cijeli ovaj kolegij ponovno koristiti
- Dva pitanja koja ćemo postavljati svakom algoritmu odsad nadalje: *što obećava, i po kojoj cijeni?*
- Plan: zagonetka → što "cijena" uopće znači → zašto ga stvarni stroj komplicira → kako uspoređivati cijene na velikoj skali → zašto "najbolje" često ne postoji → što struktura podataka zapravo kupuje → kako precizno iskazati jamstvo (*ugovor*) → kako provjeriti odgovor koji niste sami proizveli

Dio 1 — Zagonetka prije definicije (udžbenik §1.1.1–1.1.2, §1.1.4)

Zagonetka s devet kuglica

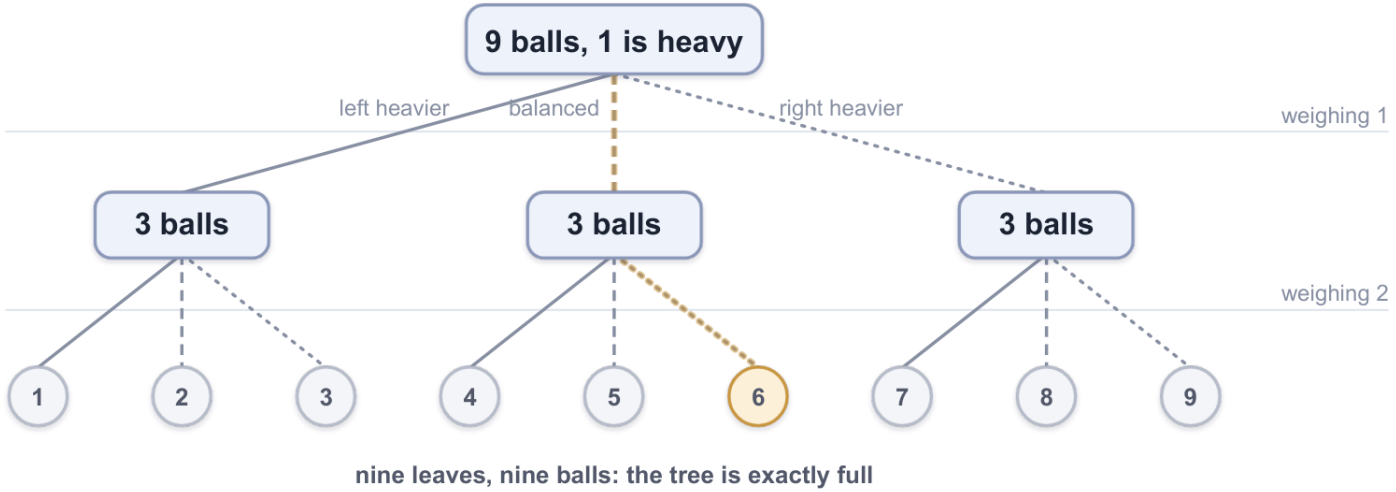
- Imate 9 kuglica, identičnih osim što je jedna neznatno teža.
- Imate vagu s dvije zdjelice: kaže Vam koja je strana teža, ili da su jednake. Ništa drugo.
- **Pitanje:** koji je najmanji broj vaganja koji uvijek pronađe težu kuglicu — i kako znate da ne možete bolje?



nine leaves, nine balls: the tree is exactly full

Odgovor: dva vaganja, nikad više

- Podijelite u tri skupine po 3.
- Izvažite skupinu A naspram skupine B. Vaga se ili nagne na jednu stranu (teža je kuglica u... čekajte, u onoj koja pokazuje teže) ili je u ravnoteži (kuglica je u trećoj skupini).
- Ponovite isti trik unutar preostale skupine od 3 kuglice.
- **2 vaganja. Uvijek. Bez iznimke.**



Opće načelo: donja ograda stabla odlučivanja

- Svako vaganje ima 3 moguća ishoda → strategija je **ternarno stablo odlučivanja** (decision tree)
- Stablo dubine *w* ima najviše 3^w listova
- Da razlikujete 9 kuglica, trebate barem 9 zasebnih listova → 3^w ≥ 9 → w ≥ 2
- **Opće pravilo:** ako svaki korak daje jedan od *b* odgovora, a zadatak ima *N* mogućih ishoda za razlikovanje, potrebno je barem log_b(N) koraka — koliko god algoritam bio domišljat.
- Udžbenik ovo zove **donja ograda protivnika / stabla odlučivanja** (adversary / decision-tree lower bound): *protivnik* je zamišljeni oponent koji na svakom koraku daje onaj odgovor koji ostavlja najviše ishoda otvorenima.

Ista ideja, negdje mnogo veća

- Binarno pretraživanje: svaka usporedba daje odgovor da/ne (b = 2) → potrebno je ≥ $\log_2 n$ usporedbi za pronalazak jednog elementa među n (točnije $\log_2(n+1)$, jer je i "nije pronađeno" odgovor — Slajd 39)
- Sortiranje temeljeno na usporedbama: mora razlikovati svih n! mogućih poredaka, koristeći samo usporedbe da/ne → potrebno je ≥ $\log_2(n!)$ usporedbi ≈ **n log n**
- Ovo je **pod koji ne može ići nijedno sortiranje temeljeno na usporedbama koje će ikad biti napisano** — uključujući ona kojih se još nitko nije sjetio

Još tri pojma iz rječnika koji će se kolegij vraćati

- **Invarijanta petlje** (loop invariant) — svojstvo koje ostaje istinito u svakoj iteraciji, koristi se za dokazivanje *ispravnosti*. (Binarno pretraživanje: "ako cilj postoji, i dalje je između lo i hi.")
- **Monovarijanta** (monovariant) — veličina koja se strogo smanjuje (ili raste) svakim korakom, koristi se za dokazivanje *terminacije*. (Svako vaganje smanjuje skup kandidata — postupak mora završiti.)
- **Podjela problema** — rješavanje velikog slučaja rješavanjem manjih kopija samog sebe; ukupna cijena je utrka između posla obavljenog pri vrhu (podjela/spajanje) i posla koji se gomila pri dnu. Koja strana je teža, ona određuje cijenu; kad su izjednačene, svaka razina košta otprilike isto, pa je cijena = posao jedne razine × broj razina. (Binarno pretraživanje je taj izjednačeni slučaj: jedna usporedba po razini, zadrži se samo jedna polovica → cijena raste samo s brojem razina, $\log_2 n$. Merge sort je također izjednačen, ali na većoj skali: oko n posla po razini, $\log_2 n$ razina → $n \log_2 n$.) Uobičajeno ime za ovu "utrku": **master teorem**.

Dio 2 — Što algoritam zapravo troši (udžbenik §1.2.1–1.2.2)

Što je "resurs"?

- **Resurs** = sve što proračun troši i što se može potrošiti do kraja
- Koji resurs prvi ponestane ovisi o *stroju i zadatku* — ne o samom algoritmu
- Resursi koje će ovaj kolegij brojati, otprilike redoslijedom pojavljivanja:
  - **Vrijeme** — poduzeti koraci (zadano; "brže" gotovo uvijek znači ovo)
  - **Prostor** — koliko treba biti u memoriji u jednom trenutku (ponekad je prostor jedina granica: sažetak koji stane u memoriju pobjeđuje točan odgovor koji ne stane — cijeli argument za skice (sketches) kasnije)
  - **Prijenosi blokova** (block transfers) — putovanja do nečega sporijeg (disk, mreža); važan je broj *putovanja*, ne broj pregledanih bajtova (zbog toga postoji B-stablo, poglavlje o pretraživanju)
  - **Usporedbe / provjere / upiti** — jedna specifična operacija, broji se jer je ona skupa. Brojanje jedne specifične operacije ono je što uopće omogućuje dokaz donje ograde — fiksira koliko algoritam smije naučiti po koraku
  - **Slučajni bitovi, poruke/propusnost, rad i raspon** (work & span) — specijalizirane valute za randomizirane, distribuirane i paralelne algoritme (kasnija poglavlja)

Dvije stvari koje vrijedi zapamtiti o resursima

1. **Resursi se trguju jedni za druge.** Trošiti prostor da se uštedi vrijeme (indeks) najstariji je potez u knjizi. Trošiti vrijeme da se uštedi prostor sljedeći je najstariji. Rijetko postoji dizajn koji pobjeđuje na svakoj osi odjednom — a dizajn za koji se čini da pobjeđuje obično je mjeren na premalo osi.
2. **Svaka tvrdnja o cijeni je tvrdnja unutar modela — a model je pretpostavka.** Reći da je operacija "O(1)" znači "košta isto bez obzira koliko podataka postoji" — a to nikad nije bez pretpostavki o tome što se broji, a što zanemaruje.

Dio 3 — Zašto jedan korak nije jedan korak (udžbenik §1.2.3–1.2.6)

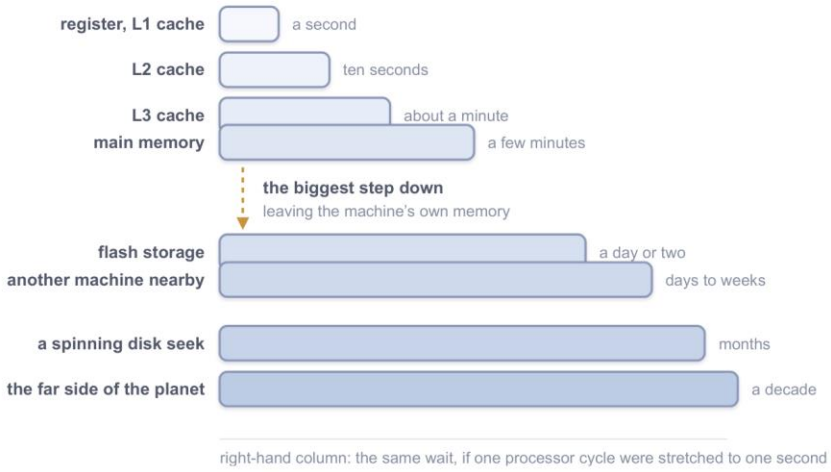
Prikladna laž RAM modela

- RAM model kaže: čitanje vrijednosti iz memorije košta **jedan korak**, uvijek, bez obzira gdje se ta vrijednost nalazi.
- Nijedan stvarni stroj nikad nije tako radio — i, prema udžbeniku, nikad neće. Nije riječ o inženjerskom propustu. **Fizika i ekonomija to obje prisiljavaju.**

Ljestvica memorijske hijerarhije

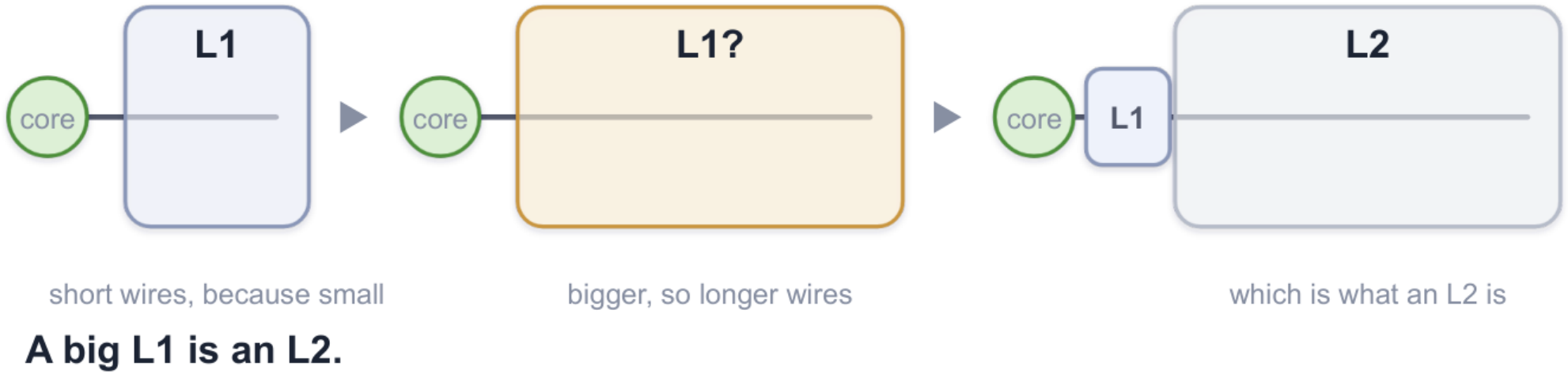
Gdje su podaci	Koliko dugo traje	naspram 1 ciklusa	Kad bi ciklus trajao 1 sekundu
Registar, L1 predmemorija	ciklus do nekoliko ciklusa	×1–×5	sekunda do nekoliko sekundi
L2 predmemorija	nekoliko ns	×10	~10 sekundi
L3 predmemorija	desetci ns	$\times 10^2$	oko minute
Glavna memorija (DRAM)	~100 ns	$\times 10^2$–10^3	nekoliko minuta
Flash (SSD)	desetci µs	$\times 10^5$	dan ili dva
Drugi stroj, ista zgrada	stotine µs	$\times 10^5$–10^6	dani do tjedni
Pristup vrtećem disku (seek)	milisekunde	$\times 10^7$	mjeseci
Drugi stroj, druga strana planeta	~100 ms	$\times 10^8$–10^9	oko desetljeća

- **Poanta nije u točnim brojkama — nego u obliku.** Brojke će zastarjeti; poredak i razmaci reda veličine neće.
- **Najveći ponor je napuštanje stroja u potpunosti** (RAM → flash/mreža), ne kretanje između razina predmemorije.



Zašto to oblikuje fizika

- **Udaljenost je vrijeme.** Svjetlost putuje ~10 cm po ciklusu takta (manje u žici). Sve što je dalje *ne može* odgovoriti unutar jednog ciklusa — točka, bez obzira na inženjering.
- **Na čipu su žice usko grlo, ne tranzistori.** Tranzistori postaju sve brži; duge žice tvrdoglavo ne.
- **Brzo, gusto, hladno — birajte manje od tri.** Brza memorija (SRAM) je velika po bitu. Gusta memorija (DRAM) je sporija. Brzi sklopovi rade vruće, a toplina mora izaći kroz površinu, pa brzi sloj mora ostati fizički malen.
- **Zaključak: ne možete izgraditi "veliku L1 predmemoriju".** Velika L1 predmemorija, do trenutka kad postane velika, jednostavno je postala L2.

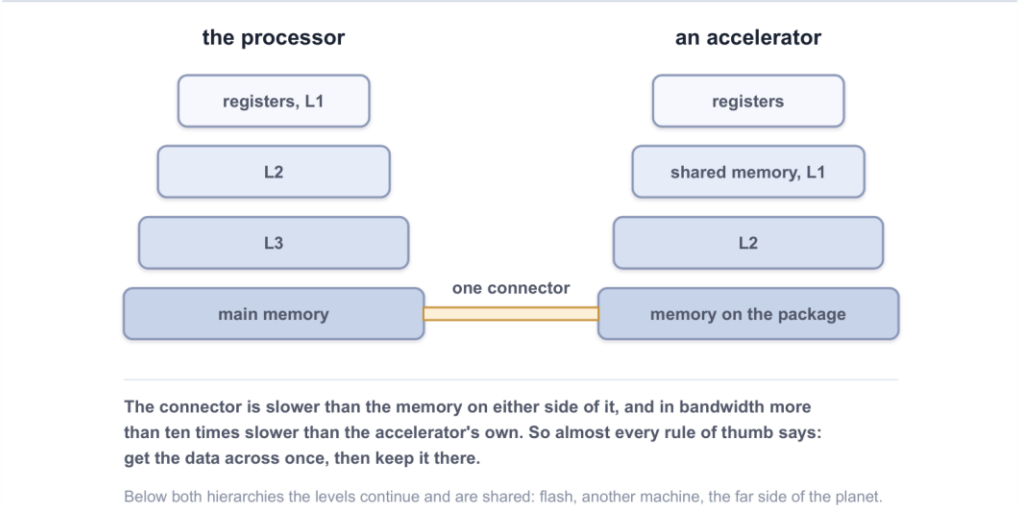


Zašto to prisiljava i ekonomija

- Cijena po bajtu pada otprilike jednako brzo kao što raste latencija, korak po korak — nije slučajnost, ista fizička ograničenja gledana s cjenovne strane.
- Stroj izgrađen isključivo od najbržeg sloja: apsurdno skup, sitan kapacitet.
- Stroj izgrađen isključivo od najsporijeg sloja: jeftin, ogroman, neupotrebljiv.
- **Optimalna raspodjela proračuna uvijek je: malo brze robe, mnogo jeftine robe, plus shema koja ono što uskoro trebate drži pri vrhu.** Nitko nije birao taj oblik — proizlazi iz minimiziranja troška uz stvarna ograničenja.

Ni GPU-ovi ovome ne izmiču

- GPU ima vlastitu ljestvicu: registri → dijeljena memorija/L1 → L2 → memorija velike propusnosti unutar paketa → preko konektora do memorije domaćina → vlastita ljestvica domaćina.
- **Konektor je usko grlo** — sporiji je od memorije s obje svoje strane, a po propusnosti više od deset puta sporiji od vlastite memorije akceleratora.
- Gotovo svako pravilo palca u programiranju akceleratora kaže: *prebacite podatke jednom, pa ih tamo zadržite.*
- Neki noviji dizajni stavljaju procesor i akcelerator u jedan paket s jednom dijeljenom memorijom — tamo se "most" smanjuje ili nestaje, a program onda odlučuju razine iznad njega.
- GPU-ovi ne čine pristup memoriji brzim — skrivaju čekanje pokretanjem ogromnog broja niti i prebacivanjem među njima dok neke čekaju.



Što memorijska hijerarhija znači za dizajn algoritama

- Dva algoritma s identičnim brojem operacija mogu se razlikovati za redove veličine u stvarnom vremenu.
- Obično je razlika: jedan prolazi kroz memoriju po redu; drugi skače nepredvidivo.
- **Broj operacija nikad nije bio cijela priča** — *gdje* podaci žive, i koliko se puta moraju pomaknuti, ostatak je priče.
- Zato će kolegij ponekad brojati **prijenose blokova** (block transfers) umjesto (ili uz) sirove operacije — to je model koji objašnjava zašto B-stablo uopće postoji.
- Treći model: brojanje prijenosa *bez* poznavanja veličine bloka (algoritmi neovisni o predmemoriji, Slajd 42). A struktura koja raspršena pisanja pretvara u sekvencijalna — poput LSM-stabla — odgovara na isti pritisak s druge strane.

Dio 4 — Kojom brzinom raste cijena (udžbenik §1.3)

Big-O, Big-Omega, Big-Theta — jednostavne definicije

- **O(g(n))** — raste *ne brže od* g. Gornja ograda: obećanje da nećete platiti više od ovoga.
- **Ω(g(n))** — raste *ne sporije od* g. Donja ograda: obično loša vijest — ništa Vas ne vodi ispod ovoga.
- **Θ(g(n))** — oboje istovremeno: rast je točno određen.
- Precizno (nova verzija knjige dodaje formalnu definiciju): $f(n) = O(g(n))$ znači da postoje konstante $c > 0$ i n_0 takve da je $f(n) \leq c \cdot g(n)$ za svaki $n \geq n_0$; Ω okreće nejednakost, Θ traži obje.
- Sva tri **zanemaruju konstantne faktore** — O(n) ništa ne govori o tome traje li korak nanosekundu ili minutu. Govori da udvostručenje n otprilike udvostručuje ukupno vrijeme.
- **Konstante odlučuju koji program isporučujete danas. Rast odlučuje koji će još raditi sljedeće godine.**

Tablica stopa rasta, i kako se koja "osjeća"

Rast	Kad se n udvostruči	n koje je isplativo u 1 sekundi*	Osjeća se kao
1	ne mijenja se	bilo koje	uključivanje prekidača za svjetlo
log n	+1 korak	sve što stane u memoriju	pogađanje broja 1–1.000.000 prepolovljivanjem raspona (dvadeset pitanja)
√n	raste za oko 40%	10^{18}	ormar s √n ladica po √n spisa: pregledaj oznake ladica, pa jednu ladicu
n	udvostruči se	10^{9}	čitanje svakog imena na popisu gostiju
n log n	malo više od udvostručenja	4×10^{7}	sortiranje špila spajanjem hrpica
n^2	učetverostruči se	30.000	svatko se rukuje sa svakim
n^3	raste osam puta	1.000	ista rukovanja, ali svako mora posvjedočiti svaki gost
2^n	kvadrira se	30	odluka poziv/nema poziva za svakog gosta, provjera svakog mogućeg popisa
n!	mnogo gore od kvadriranja	12	posjedanje svih gostiju na svaki mogući način

*proračun: stroj koji radi milijardu koraka u sekundi, jedna sekunda strpljenja — brojke će zastarjeti, oblik neće.

- **Skok nakon n^3 nije razlika u stupnju.** Svaki redak do n^3 znači "usporava kako podaci rastu". 2^n i n! znače "u nekom trenutku jednostavno prestaje raditi" — stroj 10× brži pomakne zid za tri elementa kod 2^n, a za jedan kod n!.

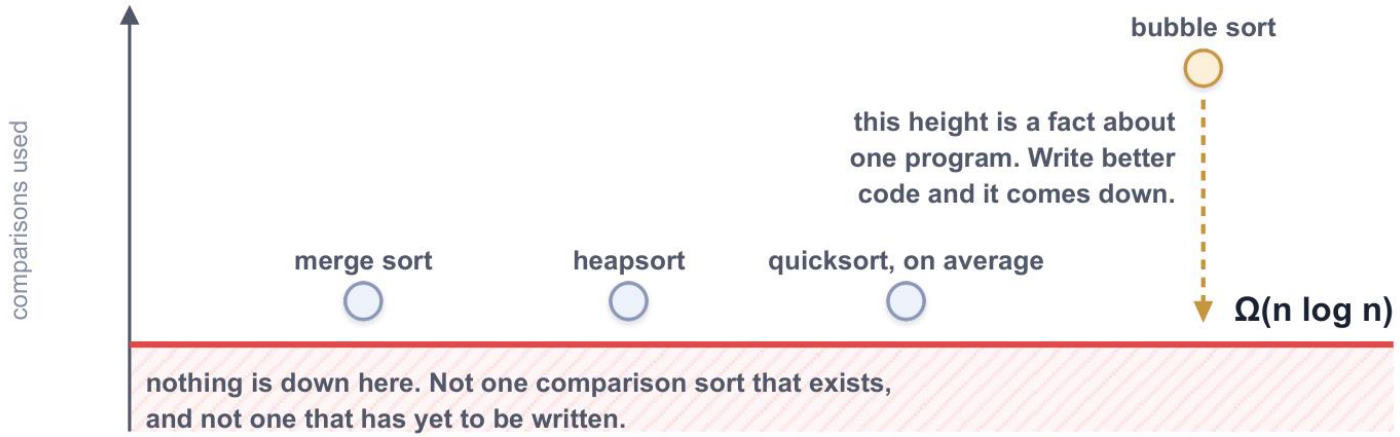
O kojem izvođenju govorimo? Pet načina brojanja cijene

- **Najgori slučaj** (worst case) — najskuplji ulaz te veličine. Zadano u ovom kolegiju — jedino koje vrijedi za *svaki ulaz pri svakom izvođenju*.
- **Prosječni slučaj** (average case) — prosjek preko ulaza, uz neku *iskazanu* distribuciju. Pouzdano samo koliko i ta distribucija — a ona je obično pretpostavka koja pukne.
- **Amortizirano** (amortized) — prosjek preko *niza operacija* nad jednom strukturom. Kaže da je dugi niz ukupno jeftin — ništa ne govori o onoj jednoj operaciji koju upravo čekate.
- **Očekivano** (expected) — prosjek preko *vlastitih* nasumičnih izbora algoritma, za fiksni ulaz. Slučajnost je Vaša, ne protivnička.
- **S velikom vjerojatnošću** (with high probability, w.h.p.) — uopće nije prosjek: vrijedi osim na iščezavajuće malom udjelu ishoda (obično najviše 1/n^c za konstantu c). Ograničava *koliko često* je izvođenje loše, ali ne i *koliko je loše* to rijetko loše izvođenje. **Nije jača verzija "očekivanog", nego drugačije obećanje.**

Obećanje	Što varira	Što je zajamčeno
najgori slučaj	ulaz; uzima se najgori te veličine	granica, na svakom ulazu i svakom izvođenju
prosječni slučaj	ulaz, iz iskazane distribucije	granica u prosjeku, *ako* ulazi stvarno prate tu distribuciju
amortizirano	mjesto u nizu operacija nad jednom strukturom	ukupno za niz, ne za pojedinu operaciju
očekivano	vlastiti nasumični izbori algoritma, za fiksni ulaz	granica u prosjeku preko nasumičnih izbora, za svaki ulaz
w.h.p.	vlastiti nasumični izbori algoritma	granica na svim osim iščezavajuće malog udjela izbora — ne i koliko su ti loši

Najčešća pogrešna interpretacija u ovom području

- Ograda može biti o **jednom algoritmu** ("ovaj kôd treba Ω(n²) koraka" → napišite bolji kôd) ili o **cijelom problemu** ("sortiranje usporedbama treba Ω(n log n) usporedbi" → *nijedan* kôd, napisan ili tek nenapisan, ne može ovo nadmašiti).
- **Provjera zablude** (u knjizi označena kao "No-AI check"): netko kaže da ima sortiranje u O(n log n), netko drugi da bubble sort radi u O(n log n). Jedno je obično, drugo je pogrešno — koje, i koja se vrsta ograde s kojom brka? Bubble sort *nije* O(n log n): u najgorem slučaju radi oko $n^2/2$ usporedbi, Θ(n²). Ograda Ω(n log n) je pod ispod *cijele klase* sortiranja usporedbama — ništa ne govori o tome koliko visoko iznad tog poda sjedi neki konkretan algoritam.
- Miješanje "poda ispod svih" sa "stropom na jednom programu" udžbenik naziva najčešćom pogrešnom interpretacijom u cijelom području.



the line is a fact about the problem. The dots are facts about programs. Only one of those is anybody's fault.

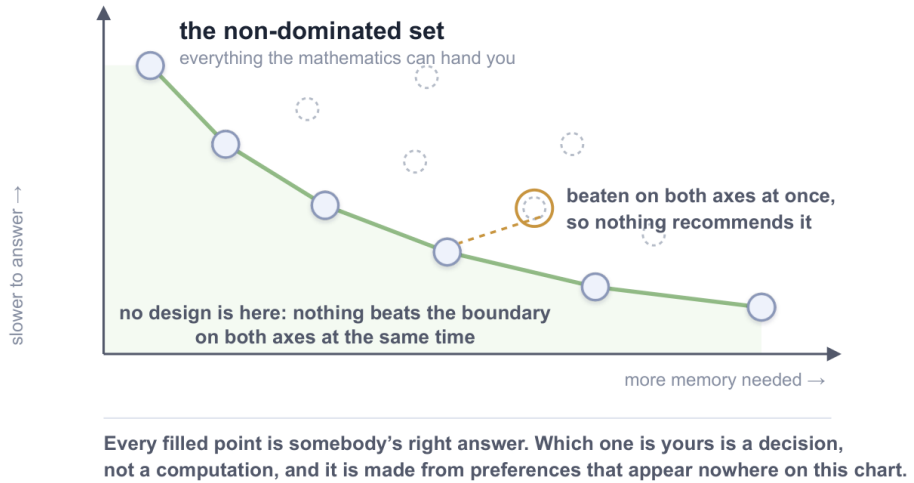
Dio 5 — Kad ne postoji jedno najbolje (udžbenik §1.4)

Ponekad "što je bolje" nema odgovor

- Dizajn A: točni odgovori, treba gigabajt.
- Dizajn B: približni odgovori, pogreška samo u jednom smjeru, stane u nekoliko megabajta.
- **Što je bolje? Potpuno ovisi o tome za što ga koristite.** Bankovna glavna knjiga: A pobjeđuje, bez rasprave. Usmjerivač koji za nanosekunde odlučuje treba li paket dodatnu provjeru: B pobjeđuje, jednako lako. Ništa se u samom *dizajnu* nije promijenilo.
- Škola Vas uči očekivati jedan broj koji treba minimizirati, jer su realni brojevi potpuno uređeni — među bilo kojim konačnim skupom kandidata "najbolje" postoji. **Dizajn nema takav broj.**

Dominacija i "nedominirani skup" (Paretova fronta)

- Dizajn A **dominira** dizajn B kad A nije lošiji od B ni na *jednoj* osi, a strogo je bolji na barem jednoj.
- Ako A dominira B, **B otpada** — na osima koje ste izmjerili, za sve, bez rasprave o preferencijama.
- Ono što ostane nakon uklanjanja svakog dominiranog dizajna zove se **nedominirani skup** (odnosno Paretova fronta): stvarni ishod usporedbe dizajna.
- **Stroj može izračunati taj skup. Sam skup ne može odabrati točku na sebi.** Svaka preostala točka je neporažena, pa nikakav izračun sam ne bira među njima — za to trebaju **preferencije**, a njih nema u podacima.
- Preferencije pomaže iskazati osoba, organizacija ili AI sustav; kad su jednom iskazane, stroj može napraviti odabir. **Iskazivanje preferencija dio je specifikacije.** Pitanja: koliko ovdje košta memorija? Koliko košta zakašnjeli odgovor, a koliko pogrešan? Tko plaća kad je sporo, i koliko često loš slučaj stvarno dođe?



Every filled point is somebody's right answer. Which one is yours is a decision, not a computation, and it is made from preferences that appear nowhere on this chart.

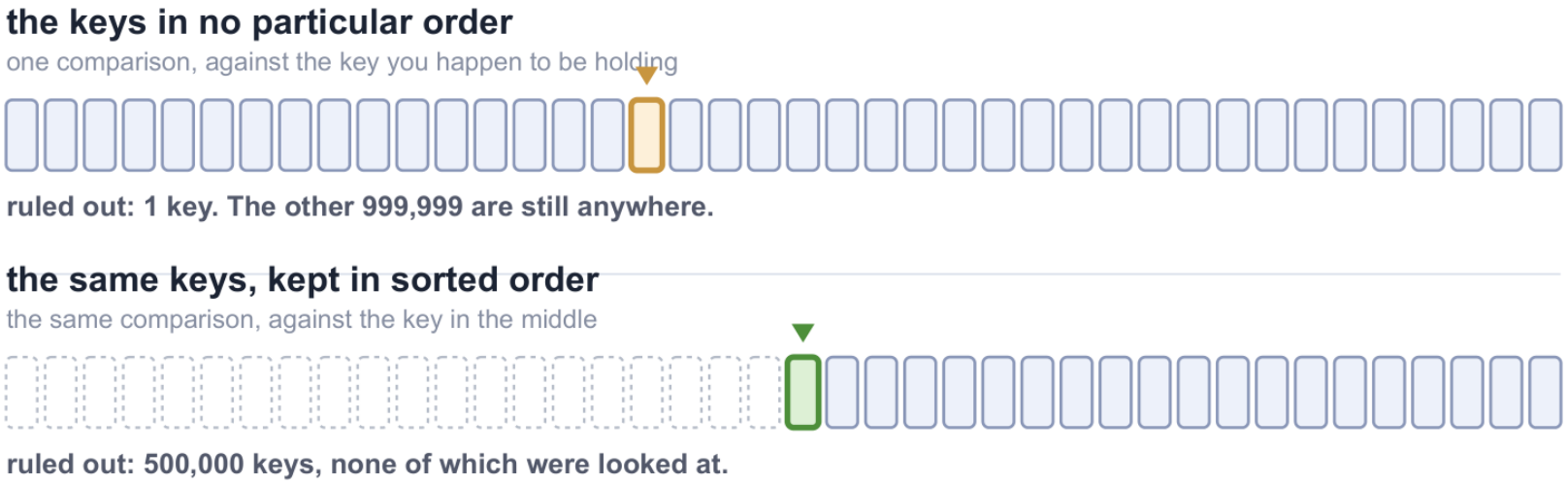
Upozorenje o nedostajućoj osi

- Osi na kojima crtate frontu i same su izbor.
- Nacrtajte memoriju naspram vremena i dobit ćete frontu za memoriju i vrijeme — ali što s: troškom rada, koliko toga puca kad zakaže, koliko dugo novom inženjeru treba prije nego sigurno može dirati sustav?
- **Dizajn koji je nenadmašiv na grafu koji ste nacrtali može biti jednostavno pogrešan na osi koju niste nacrtali.** Ovo je najčešći način na koji dobra analiza proizvede loš sustav.
- **Zapisivanje osi je ono čemu služi ugovor:** njegova *cijena* kaže koju ste os kupili, a *način otkazivanja* što obećanje ne pokriva. Os se i dalje smije izostaviti — ali samo otvoreno, napismeno, gdje netko može prigovoriti.

Dio 6 — Čemu struktura podataka zapravo služi (udžbenik §1.5.1)

Ista operacija, dvaput, potpuno različita isplata

- Milijun ključeva, **bez ikakvog reda**: jedna usporedba s ključem isključuje točno *taj jedan ključ*. Preostalih 999.999 i dalje je bilo gdje.
- Isti milijun ključeva, **sortirani**: ista usporedba, sa srednjim ključem, isključuje **pola milijuna** — od kojih niste pogledali nijedan.
- **Identična operacija. Identična cijena. Jedna od njih nauči 500.000× više.**



Što je struktura podataka, formalno

- **Struktura podataka je raspored podataka koji određene zaključke čini dostupnima jeftino** — tako da algoritam izgrađen na njoj po koraku dosegne dalje nego bi inače mogao.
- **Nije** pohrana. Nije spremnik. To je *trajni skup činjenica* na temelju kojeg smije rasuđivati svaka operacija.
- Oštrica: struktura čini neke zaključke jeftinima **odbijajući ostale.**
  - Sortirano polje: "u kojoj je polovici?" je besplatno. "Umetni još jedan element" je skupo.
  - Hash tablica: "je li točno ovaj ključ prisutan?" je besplatno. "Koji je sljedeći ključ nakon ovog?" košta sve što bi red donio.
- **Strukturu ne birate po tome što sadrži. Birate je po tome koji Vam zaključci trebaju jeftini — i kojih se možete odreći.**

Dio 7 — Dizajniranje putem invarijanti (udžbenik §1.5.2–1.5.5)

Struktura su njena svojstva, ne njen kôd

- Jeftini zaključci s prethodnog slajda dostupni su samo zato što određene činjenice o podacima ostaju istinite **trajno** — ne obično, ne "u uobičajenom slučaju", nego nakon *svake pojedine* operacije, zauvijek.
- Ovo je **invarijanta petlje** (loop invariant), promaknuta iz tehnike dokazivanja (Slajd 10) u tehniku *dizajna*.
- **Struktura podataka nije njen kôd. To je skup svojstava koje obećava održavati — a zaključci koje čini jeftinima točno su oni koje ta svojstva dopuštaju.**

Razrađen primjer: lijevo naginjuće crveno-crno stablo

Lijevo naginjuće crveno-crno stablo (left-leaning red-black tree) je točno **pet** stvari koje vrijede istovremeno:

1. Ključevi su poredani
2. Najviši čvor (korijen) računa se kao crn
3. Crvene veze uvijek naginju lijevo
4. Nikad se ne dodiruju dvije crvene veze
5. Svaki put prema dolje prolazi kroz isti broj crnih veza

- Ne trebate danas znati *zašto* ovih pet svojstava održava stablo niskim (to dolazi kasnije u kolegiju) — poanta je danas samo: **ova konjunkcija pet svojstava *jest* struktura. Sve ostalo (bit boje, rotacije, okretanja boja) je zapis**, odabran jer tih pet svojstava čini jeftinima za održavanje.
- "Standardno" crveno-crno stablo koje predaje većina udžbenika razlikuje se od ovog labavljenjem točno **dva** od ovih pet pravila: izbacite "crvene veze naginju lijevo", i oslabite "nikad se ne dodiruju dvije crvene veze" tako da zabranjuje samo crvenu vezu ispod druge crvene, a ne po jednu sa svake strane istog čvora. Dvije strukture koje u kodu izgledaju posve drugačije, a razlikuju se za točno dva obećanja. **To ne vidite uspoređujući implementacije — samo uspoređujući obećanja.**

Specifikacija: pravila i značenje

- Kad je struktura definirana invarijantama, svaka operacija prestaje biti kôd i postaje **obveza**: *ako je struktura bila valjana (svih pet pravila vrijedi) prije, valjana je i poslije* — *i** skup ključeva promijenio se točno na ovaj navedeni način.*
- To je cijela *funkcijska* specifikacija `insert`-a. Svaka implementacija koja je zadovoljava ispravan je `insert` — u bilo kojem jeziku, tko god (ili što god) ga napisao. Implementacija dolazi nakon specifikacije i može se zamijeniti.
- **Trebaju obje polovice.** Izbacite drugu i `insert` koji tiho ne radi ništa prolazi: struktura je bila valjana, i još uvijek jest.
- **Pravila** kažu koja su stabla dopuštena. **Značenje** (skup ključeva koji stablo drži) kaže koji odgovor izlazi: `insert(k)` ostavlja skup s dodanim k i ničim drugim promijenjenim.
- **Najmanji primjer:** specificirajte sortiranje kao "izlaz je poredan". Program koji ignorira ulaz i vrati praznu listu — prolazi. Sto nula — prolazi. Tek dodatak "*i izlaz je permutacija ulaza*" ih isključuje.

Odakle dolaze invarijante

- Obično iz **cilja koji je olabavljen dok ga nije moguće održavati.**
- Binarno pretraživanje želi "cilj je na `mid`" — to ne može znati do kraja. Zato drži slabije obećanje koje može obnoviti nakon svake provjere: *"cilj, ako postoji, nalazi se u [lo, hi]"*. Istinito prije prve provjere, preživi svaku, a na kraju (raspon od jednog mjesta ili prazan) — to *je* odgovor.
- Strukture se dizajniraju isto: "svaki put prema dolje ima istu duljinu" ne preživi ni jedno umetanje. "Svaki put prema dolje prolazi kroz isti broj crnih veza" preživi — uz popravak blizu promjene.
- Obećanja omeđuju **područje**: skup rasporeda u kojima struktura smije biti. Odabir područja je većina dizajna:
  - dovoljno **usko** da svaki raspored unutar njega ima željenu cijenu,
  - dovoljno **široko** da se ažuriranje koje izađe van može vratiti malim lokalnim popravkom.

Ugovori: trodijelni oblik svakog obećanja

Svaka operacija postaje obveza čim je struktura definirana svojim invarijantama. **Ugovor dizajna = njegovo obećanje, iskazano na značenju, plus tri polja** koja nosi svaka ideja za ponovnu upotrebu u ovom kolegiju:

Polje	Što je to
Pretpostavka (assumption)	uvjet pod kojim jamstvo vrijedi
Cijena (cost)	granica resursa koju dobivate *dok* ta pretpostavka vrijedi
Način otkazivanja (failure mode)	što puca kad pretpostavka ne vrijedi — što obećanje *ne* pokriva

Razrađen primjer — binarno pretraživanje kao ugovor:

- *Obećanje:* pronađi ključ ili javi da ga nema
- *Pretpostavka:* ključevi su već sortirani
- *Cijena:* oko $\log_2 n$ usporedbi, najgori slučaj
- *Način otkazivanja:* ako polje nije sortirano, prepolavljanje tiho odbacuje polovicu koja je zapravo sadržavala odgovor — bez greške, bez upozorenja, samo pogrešan rezultat



Od kartice ugovora do dizajna: sedam pitanja

Kartica ugovora *bilježi* dizajn kad je gotov. *Izrada* dizajna prolazi kroz ista polja kao pitanja, obično ovim redom:

1. Koje je **opterećenje** (workload — mješavina operacija s kojima će se dizajn suočiti), i što najčešće traži?
2. Koji je resurs **oskudan** — dakle, za što plaćate?
3. Što **nijedan dizajn ne može nadmašiti**? (Pod iz stabla odlučivanja.)
4. Koje ćete **obećanje** održavati — iskazano na *značenju*, ne samo na pravilima?
5. Koja **ideja** (primitiv) to obećanje čini jeftinim?
6. Koliko **košta**, i što **odbija pokriti**?
7. Koja bi vas **promjena opterećenja** natjerala da preispitate odluku?

Zašto dizajnirati na ovaj način? Jer pomiče neuspjeh ranije

- Pisanje koda, pa njegovo testiranje, govori Vam samo o slučajevima kojih ste se sjetili testirati.
- **Pisanje invarijante prvo govori Vam nešto bolje: mjesta na kojima ne možete jasno iskazati što želite točno su mjesta na kojima je Vaš dizajn još zbunjen.**
- **Svojstvo koje ne možete zapisati je dizajn koji niste dovršili.**
- Taj signal se pojavljuje *prije nego ijedan kôd postoji* — najjeftiniji mogući trenutak da ga uhvatite.

Dio 8 — Provjera odgovora koji niste sami proizveli (udžbenik §1.6)

Provjera je jeftinija od pronalaženja

- Specifikacija vrijedi točno onoliko koliko i provjere koje se protiv nje mogu pokrenuti — a **provjeravanje je drugačiji posao od pronalaženja**, obično mnogo jeftiniji.
- **Sortiranje:** *proizvesti* sortiranu listu košta barem ~n $\log_2$ n usporedbi (to je dokazalo stablo odlučivanja).
- *Provjeriti* listu koju vam netko preda:
  - je li svaki element ≤ sljedećeg? → jedan prolaz
  - sadrži li iste elemente kao ulaz? → još jedan prolaz s brojačem po elementu u hash tablici (+1 za svaku kopiju u ulazu, −1 za svaku u izlazu, na kraju sve nula)
- Ukupno **O(n) za provjeru naspram n log n za sortiranje** → provjera si može priuštiti da se pokreće *svaki put*, čak i kad je sorter tuđi — ili ga je napisao model — i ne možete čitati njegov kôd.
- Sitni tisak: tih O(n) je *očekivana* cijena, zbog hash tablice. Samo s usporedbama brojanje bi koštalo koliko i sortiranje.

Dvije vrste provjere, i zašto provjera mora doći odnekud drugdje

- **Očiti algoritam kao definicija.** Neki problemi imaju metodu toliko jednostavnu da joj ispravnost nije upitna: pogledaj svaki element. Može biti prespora za upotrebu — nije važno, njen posao je *definirati* ispravan odgovor na malim ulazima. Pametna metoda mora se s njom slagati. (Primjer iz knjige: prateća biblioteka nakon svakog `insert`-a i `delete`-a uspoređuje svoja crveno-crna stabla s običnim Python `set`-om. Referenca se namjerno drži naivnom: referenca koja dijeli pametnu ideju mogla bi dijeliti i njenu grešku.)
- **Odgovor koji nosi vlastiti dokaz.** Neki odgovori dolaze s nečim jeftinim za provjeru: sortirana lista uz ulaz; najkraća ruta uz udaljenost za svako mjesto, što dokazuje da kraća ruta ne postoji; kasnije u knjizi tok uz rez koji dokazuje da veći tok ne postoji. Takvi algoritmi zovu se **certificirajući** (certifying algorithms). *Vjerujete odgovoru jer ste ga provjerili, ne zato što vjerujete programu.*
- **Zašto provjera mora imati drugačije podrijetlo?** Dva programa napisana iz istog nesporazuma slažu se jedan s drugim. Ako autor sortiranja napiše i test, test napisan iz istog pogrešnog čitanja zadatka propušta iste pogrešne izlaze.

Dio 9 — Dvije veličine stvari za učenje (udžbenik §1.7)

Komponente naspram primitiva

- Gotova struktura podataka (B-stablo, red, predmemorija) je **komponenta** — posežete za njom kao za bibliotekom. Korisna je, ali već je odgovor na jedno specifično pitanje, s ugrađenim kompromisom.
- Male ideje od kojih je svaka struktura *izgrađena* — one koje se ponovno pojavljuju na mjestima bez očite veze s onim gdje ste ih prvi put upoznali — su **primitivi** (primitives). (Primjeri iz *ovog* predavanja, kako ih navodi nova verzija udžbenika: argument prebrojavanja koji je riješio devet kuglica stavio je i pod ispod svakog sortiranja usporedbama i ispod binarnog pretraživanja; obećanje koje binarno pretraživanje drži ispravnim iste je vrste kao činjenice koje definiraju crveno-crno stablo.)
- **"Popis struktura je nešto što treba zapamtiti. Ideje od kojih su sačinjene nešto su s čime treba misliti."**
- Primitivi su ujedno i **riječi kojima se pišu specifikacije**: ugovor koji kaže "sortirano", "balansirano" ili "amortizirano" precizan je točno onoliko koliko su precizne te riječi.

Dio 10 — Kako udžbenik provjerava sam sebe (stara verzija udžbenika, §3.8 — u novoj verziji odjeljak ne postoji)

Provjera koja ne može pasti nije provjera

> **Napomena o izvoru:** Nova verzija poglavlja (pogl. 1) potpuno je izbacila odjeljak "How this book checks itself" (stari §3.8). Priča o 276/400 stabala, Python/Rust/Lean i brisanju u BST-u više nije u udžbeniku koji studenti čitaju. U novoj verziji ostalo je samo: (a) usporedba crveno-crnih stabala s običnim `set`-om (Slajd 36), (b) mutacijska analiza kao natuknica u "frontier map" (Slajd 42), (c) nova natuknica "Specification synthesis" (Slajd 42). Preporuka: slajd izbacite, ili ga ispričajte kao anegdotu bez upućivanja na knjigu.

- Zapisati invarijantu ne čini je istinitom. Zato se pri svakoj izgradnji knjige ispisani kôd kopira sa stranice i pokreće protiv prateće biblioteke (`algodesign`); izgradnja pada ako se ne slažu.
- **Zašto:** tiskana verzija rutine za okretanje boja (color flip) bila je pogrešna — pokvarila je **276 od 400** nasumično izgrađenih stabala. Biblioteka je cijelo vrijeme imala ispravnu verziju (91 prolazni test). Problem je bila stranica, ne biblioteka.
- **Slaganje nije dokaz:** biblioteka postoji u Pythonu, Rustu i Leanu — sva tri su se slagala, svi dokazi su prolazili, a brisanje u BST-u ipak je bilo pogrešno (sva tri iz istog pogrešnog čitanja). Uhvatila ga je tek usporedba s udžbenicima — *referenca drugačijeg podrijetla*.
- **Mutacijska analiza** (mutation analysis): namjerno pokvarite kôd i provjerite primjećuje li provjera. Harness knjige dobiva šest namjerno pokvarenih verzija i izgradnja pada ako ne uhvati svaku.
- **"Provjera bez zapisa o tome što je uhvatila je provjera koju nitko nije provjerio."**

Dio 11 — Povezivanje svega (udžbenik §1.1.3, §1.8–1.9 + zaključak)

Ideja stabla odlučivanja, još jednom, na tri načina

Argument prebrojavanja ishoda s početka predavanja (Slajdovi 6–9) potez je koji se ponavlja — "tvornica certifikata" (certifikat = dokaz koji možete sami provjeriti): pretvara *"ne vidim kako bih mogao bolje"* u *"nitko ne može, evo dokaza."*

- **Vaganje kuglica:** podjela na 3 → grananje na tri dokazano je optimalno
- **Sortiranje usporedbama:** brojanje svih n! poredaka → ograničava svako moguće sortiranje usporedbama odjednom
- **Pretraživanje sortiranog polja:** svaka usporedba je da/ne, a postoji n+1 mogućih odgovora (n pozicija + "nije pronađeno") → potrebno je $\geq \log_2(n+1)$ usporedbi
- Certifikat radi i na **odgovorima**, ne samo na ogradama: donja ograda je certifikat da nitko ne može bolje; sortirana lista uz ulaz je certifikat da je *ovaj* odgovor ispravan (Slajdovi 35–36).

Rekapitulacija: velike ideje s današnjeg predavanja

1. Algoritam je **obećanje plus cijena** — ispravnost i cijena, iskazane zajedno, za svaki ulaz.
2. **Stopa rasta, ne sirova brzina, odlučuje koje metode prežive na velikoj skali** — a donja ograda može biti činjenica o cijelom *problemu*, ne samo o jednom programu.
3. **"Jedan korak" skriva model.** Stvarni strojevi imaju memorijsku hijerarhiju; jednolika cijena RAM modela prikladna je fikcija s fizičkim i ekonomskim razlozima iza sebe.
4. **Često ne postoji jedno "najbolje" rješenje** — samo nedominirani skup, a odabir točke na njemu traži preferencije koje dolaze izvan podataka i koje treba zapisati kao dio specifikacije.
5. **Struktura podataka je raspored koji određene zaključke čini jeftinima odbijajući ostale** — a cijena algoritma zapravo je činjenica o strukturi ispod njega.
6. **Specifikacija = pravila + značenje**, a **ugovor** — obećanje iskazano na značenju, plus pretpostavka, cijena i način otkazivanja — način je da jamstvo iskažete dovoljno precizno da se na njemu može graditi.
7. **Provjeriti je jeftinije od pronaći** — a provjera vrijedi samo ako dolazi iz drugačijeg izvora od onoga što provjerava.

Pitanja za sljedeći put

1. **Vj. 1 (vaga, ~20 min)** Neispravna kuglica sada može biti teža *ili* lakša, a ne znate koja — zašto razlikovanje 2n mogućnosti mijenja ogradu, i koliko vaganja treba 12 kuglica?
2. **No-AI provjera** Kolega tvrdi da ima sortiranje temeljeno na usporedbama koje radi u O(n) vremenu. Bez pokretanja: moguće ili ne — i koju bi točno donju ogradu moralo prekršiti? (Ako bi moglo biti moguće — koju pretpostavku te ograde krši?)
3. **Vj. 3 (cjenovni modeli, ~15 min)** Imenujte cjenovni *model* koji čini svaku od ovih tvrdnji istinitom ili lažnom: "hashiranje je O(1)." "Umetanje u dinamičko polje je O(1)." "Binarno pretraživanje na disku je brzo."
4. **Vj. 4 (ugovori, ~15 min)** Za binarno pretraživanje, hash tablicu i mali da/ne sažetak imenujte pretpostavku i način otkazivanja. Koji od njih otkazuju *tiho*, i kako biste to primijetili?
5. **Vj. 5 (amortizacija, ~20 min)** Dinamičko polje udvostruči veličinu kad se napuni. Pokažite da n umetanja ukupno košta O(n) — dakle amortizirano O(1) po umetanju — iako jedno umetanje povremeno može koštati O(n).
6. **Vj. 6 (specifikacije, ~25 min)** Tri specifikacije za "ukloni najmanji element iz kolekcije": (a) kolekcija je nakon toga za jedan element kraća; (b) vraćeni element nije veći ni od čega što je ostalo; (c) vraćeni element **bio je u kolekciji** i nije veći ni od čega što je ostalo, **i** kolekcija nakon toga sadrži točno one elemente koje je imala prije, minus **jednu kopiju** tog elementa. Za (a) i (b) napišite pogrešan `remove_min` koji ih zadovoljava. Zašto (c) ne dopušta nijedan pogrešan? I na kraju: što bi (c) trebala reći kad je kolekcija prazna?
7. **Vj. 7 (specifikacije, ~10 min)** U specifikaciji sortiranja sa Slajda 30 izbacite "poredan" umjesto "permutacija ulaza". Imenujte, u jednom retku, pogrešno sortiranje koje oslabljena specifikacija sad propušta.
8. **Projekt 1 (~30 min) — Predajte to AI-u.** Dajte AI asistentu specifikaciju (b) iz Vj. 6 doslovno i tražite implementaciju. Provjerite je prema (c). Vjerojatno prolazi — recite *što* je popunilo rupu koju je (b) ostavila, i zašto se na to ne biste mogli osloniti za operaciju koju još nitko nije imenovao.
9. **Projekt 2 (~45 min, programerski)** Napišite provjeru izlaza sortiranja: poredano *i* permutacija ulaza. Izmjerite je naspram Pythonovog `sorted` na milijun nasumičnih cijelih brojeva — kako njeno vrijeme raste s n? (Vj. 2 — dokaz ograde $\Omega(n \log n)$ iz $\log_2(n!)$ Stirlingovom aproksimacijom, označena "hard" — izostavljena za manje naprednu publiku.)

(Opcionalna najava) Kamo ovo dalje vodi

- **Algoritmi neovisni o predmemoriji** (cache-oblivious algorithms) — prilagođeni dobrom ponašanju u prijenosu blokova *bez* unaprijed poznate veličine bloka (noviji radovi uspoređuju ih s algoritmima koji se prilagođavaju dok se raspoloživa memorija mijenja). Posegnite za ovim kad konstante i lokalnost podataka dominiraju.
- **Paralelni modeli rada i raspona** (work-span) — kad mnogo procesora radi odjednom, "koliko koraka" prestaje biti jedan broj; trebate i ukupan rad *i* najduži neizbježni lanac ovisnih koraka (raspon, span).
- **Certificirajući algoritmi** (certifying algorithms) — uz svaki odgovor vraćaju dokaz da je ispravan, koji provjerava mali, zasebno pouzdan provjerivač. Posegnite za ovim kad program dolazi odnekud gdje ga ne možete pregledati.
- **Provjere pod optimizacijom** — kad nešto optimizira protiv Vaše provjere (npr. AI agent koji "popravlja kôd dok testovi ne prođu"), pronaći će rupe u provjeri. Znak: rezultat koji raste dok ono što mjeri ne raste. Pokretanje testova na namjerno pokvarenim kopijama programa (mutacijska analiza) standardni je način testiranja testova.
- **Sinteza specifikacija** (specification synthesis) — *novo u udžbeniku:* AI sustavi koji iz primjera ili zahtjeva na prirodnom jeziku skiciraju specifikacije, invarijante i testove. Opasnost: generirani provjerivač zapiše isti nesporazum kao generirani program, pa se njih dvoje slažu. Znak: program i testovi koji su nastali iz istog prompta i nikad se nisu razišli. Kad generator piše i suca, tražite neovisne dokaze.